Convolutional Networks and Transfer Learning (Lab 5)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, October 21, 2026

Instructor: Prof. Jongmin Lee

How to use this note:

- Read it before the lab session. The lab will not repeat this material.
- Covered: what a convolution layer computes and how large its output is, parameter sharing, how sizes change as layers stack, freezing a pretrained backbone and replacing the head, and controlled comparison when labels are scarce. The emphasis is on what each method assumes and where it breaks.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab05.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy (grading and AI use) was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Compute the output size of a convolution layer from the input size, kernel size, stride, and padding. (Task 1)
2. Explain why a convolution layer has far fewer parameters than a dense layer, in terms of locality and weight sharing. (Task 2)
3. Track the size and channel count of the feature map through stacked convolutions and pooling, and derive the flattened dimension yourself. (Task 2)
4. Explain what changes when you load a pretrained backbone, freeze it, and attach a new head. (Task 3)
5. Compare a from-scratch model against a transfer model under controlled conditions when labels are scarce. (Task 4)

2 Background: an image is not a vector

Every network you built through Lab 4-2 took its input as one flat vector. Treat one photograph from this lab that way and a 32 by 32 grid becomes a 1024-dimensional vector. An edge on a part is a place where a bright pixel sits next to a dark one, but once the image is flattened those two pixels are components 1 and 33, and the fact that they were neighbours is gone.

A convolution layer keeps that information. It slides a small grid of weights across the image one step at a time, multiplying and adding, and it uses the same weights at every position. An edge detector learned in the top left still works in the bottom right.

The data for this lab is 1,200 grayscale photographs, 32 by 32, of four part shapes, and only 240 of them carry a label you may train on. The scenario is a production line that started recently, so that is all anyone has annotated. A network trained from scratch on 240 images learns something, but 240 examples are little to learn edges and curves

from. Meanwhile there are weights from a model pretrained on 9,000 photographs of six *other* part shapes from the old line. The parts differ; finding edges and curves does not. Taking the front of that model, freezing it, and training only the final classification layer for the four new shapes is what *transfer learning* means in this lab.

Said plainly:

- The backbone here is a scale model of real transfer learning. In practice you would use a model with tens of millions of parameters pretrained on a million ImageNet images. This backbone has 5,888 parameters and saw 9,000 images. It was shrunk so that the whole lab runs on CPU in well under ten minutes with no download. The procedure and the reasoning are the same at either scale.

The four steps of Lab 5:

- **Output size:** compute it from the input size, kernel, stride, and padding. (Task 1)
- **Build the network:** convolution, ReLU, pooling three times, then flatten and one linear layer. (Task 2)
- **Freeze and replace:** load the pretrained weights, freeze the front, attach a new head. (Task 3)
- **Controlled comparison:** same split, same seed, same number of epochs. (Task 4)

3 Key concepts

3.1 What a convolution layer computes

A *convolution* layer slides a small grid of weights over the input, multiplying element by element and summing. That grid is the *kernel*, and the grid that comes out is the *feature map*. Three settings decide how large the output is: kernel size, stride, and padding (Figure 1).

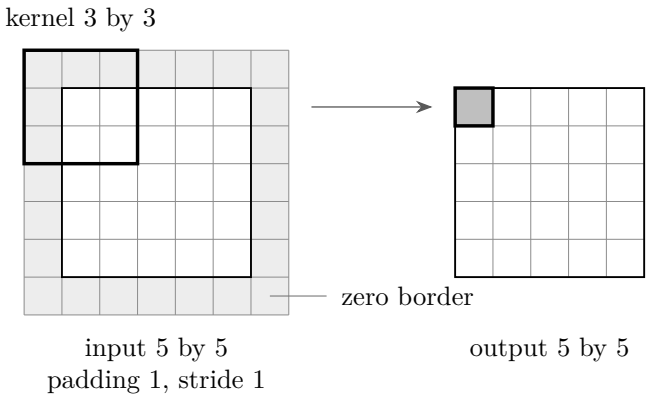


Figure 1. Where the kernel sits and the output cell it makes. The heavy 3 by 3 window produces the one shaded cell on the right. The number of positions the window can occupy is the output size.

- **Kernel size:** the input region one output cell looks at. This lab uses 3 by 3.
- **Stride:** how many cells the kernel moves at a time. A stride of 2 roughly halves the output.

- **Padding:** how many rings of zeros go around the border. A 3 by 3 kernel with padding 1 keeps the output the same size as the input.

The output-size formula (Task 1):

Spec (Task 1)

`out = floor((L + 2*padding - kernel_size) / stride) + 1`

Height and width are computed separately. Return a tuple of two plain Python ints.

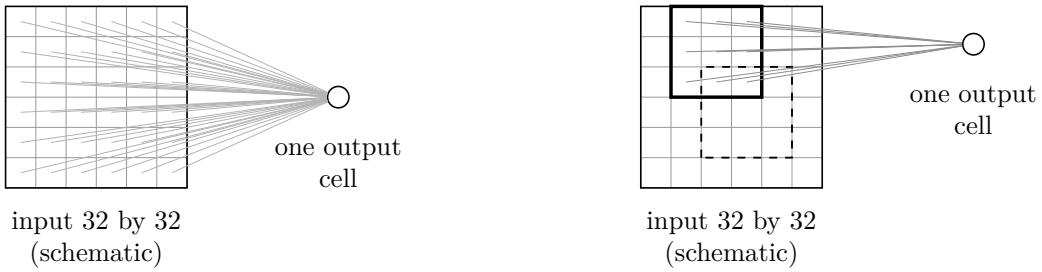
The numerator is the last position the left edge of the kernel can occupy; dividing by the stride and adding one counts the positions available. The division floors, and leftover cells are dropped. The same formula applies to max pooling.

Assumptions and limits:

- This is the formula for dilation 1. Every layer the skeleton and the hidden tests use is at its default.
- If the kernel is larger than the input the numerator goes negative and the output size drops to zero or below. That happens when you stack layers until the feature map is too small.

3.2 What parameter sharing saves

A dense layer multiplies the whole input to produce one output cell. A convolution layer looks only at the nine cells its kernel covers, and it reuses those nine weights at every position. That is *parameter sharing*, and counting the connections that reach one output cell shows how large the difference is (Figure 2).



dense layer
$1024 \times 8192 + 8192 = 8{,}396{,}800$ weights

convolution layer (8 channels)
$1 \times 8 \times 3 \times 3 + 8 = 80$ weights

Figure 2. How a dense layer and a convolution layer connect. Both produce an output of the same size, and their weight counts differ by more than a factor of a hundred thousand. The dashed window on the right is the same kernel at another position; the grids are schematic.

Producing an output of the same size:

- A dense layer connects one output cell to all 1024 input cells, so an 8-channel 32 by 32 output costs 8,396,800 weights. The convolution looks at 9 cells per output and reuses the same 9 weights everywhere, so it costs 80. A factor of 104,960.

- What you save is not only storage. The fewer values there are to fit, the less data you need to fit them, and this lab has 240 labels.

Assumptions and limits:

- Sharing pays off only when **the same pattern means the same thing wherever it appears**. In photographs an edge is an edge anywhere, so it holds.
- It breaks on data where position carries meaning. Running a convolution over a table whose columns are unrelated variables buys nothing.

3.3 Stacking layers changes both size and field of view

The network in Task 2 repeats convolution, ReLU, and max *pooling* three times, then flattens and adds one linear layer. Table 1 tracks the size and channel count at each stage.

Table 1. Layer-by-layer output and parameters of the Task 2 network. Convolution with padding 1 preserves the size; only pooling halves it. The total of 7,940 is everything that moves when you train from scratch.

Stage	after conv	after pool	parameters
input		$1 \times 32 \times 32$	0
1: conv 8 ch, ReLU, pool	$8 \times 32 \times 32$	$8 \times 16 \times 16$	80
2: conv 16 ch, ReLU, pool	$16 \times 16 \times 16$	$16 \times 8 \times 8$	1,168
3: conv 32 ch, ReLU, pool	$32 \times 8 \times 8$	$32 \times 4 \times 4$	4,640
flatten		512	0
head: `Linear(512, 4)`		4	2,052
total			**7,940**

What to read out of Table 1:

- **Size:** compare the two middle columns and you see the convolution preserving the size. Only pooling shrinks it, halving 32 to 16 to 8 to 4.
- **Channels:** what is lost in size is gained in channels, 8 to 16 to 32. The representation moves from fine detail in a small area towards counting more kinds of pattern.
- **Flattened dimension:** $32 \times 4 \times 4 = 512$. That value is `FEATURE_DIM` and the input dimension of the head. Get it wrong and the linear layer raises a shape error.
- **Receptive field:** the field of view of one output cell grows from 3 to 22, so one cell of the last feature map sees a 22 by 22 region of the input. In this data most parts fit inside 22 px, so the last layer usually sees a whole part at once.

Assumptions and limits:

- Flattening only works when the input is always 32 by 32. Change the size and the head's input dimension no longer matches.
- Max pooling keeps only the largest value in each window. That buys robustness to small shifts and costs precision about location.

3.4 Freeze the backbone, replace the head

A trained convolutional network splits into two parts. The stack of convolutions turns an image into a 512-dimensional feature vector, and the linear layer turns that vector into class scores. The front is the *backbone*, the back is the head.

- The backbone learns edges, thicknesses, and orientations, which stay useful when the subject changes. The head learns which pattern gets which class name in this particular task, and it is worthless once the task changes.
- So you take the backbone out of the shipped `data/backbone_shapes6.pt` and drop the head. The saved head is a 6×512 matrix for six classes and does not even fit the four classes here.
- Setting `requires_grad = False` on the loaded backbone parameters is called *freezing*. No gradient is computed for them, so they never reach the optimizer and never change during training.

In a frozen model the forward and backward passes do not cover the same span (Figure 3).

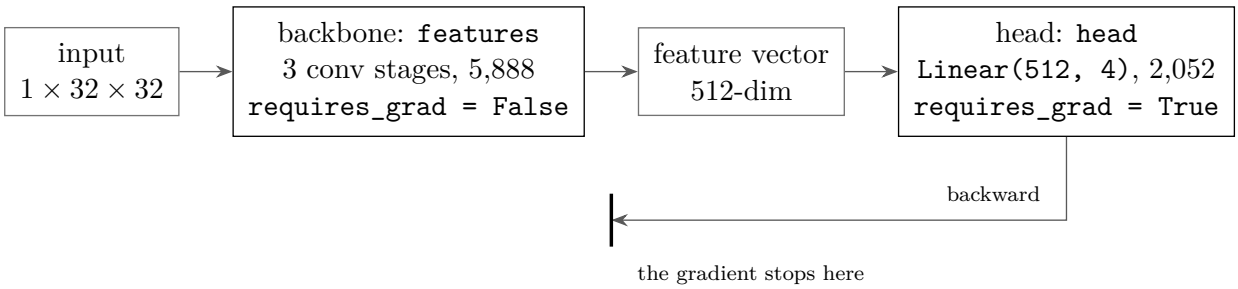


Figure 3. A frozen backbone and a trainable head. The forward pass runs left to right through everything; the backward pass stops at the head. All 5,888 backbone weights hold the same values before and after training.

What freezing actually changes:

- **The size of the optimizer:** `make_optimizer` in the skeleton collects only parameters whose `requires_grad` is true. After freezing, that is the head's 2,052 out of 7,940.
- **Speed and room to overfit:** backpropagation ends at the head, so updates are fast, and fitting 2,052 values on 240 images is easier than fitting 7,940. The cost is that the backbone never adapts to this task. With enough labels, fine-tuning the backbone as well is usually better.

Assumptions and limits:

- The pretraining task and this task are assumed to **share low-level patterns**. Here both are part photographs from the same camera, so they do.
- If the two tasks are far apart, the frozen features may already have discarded what this task needs. Using a backbone pretrained on satellite imagery for chest X-rays is that case.
- A backbone carries the biases of its pretraining data, and whoever receives only the weights cannot inspect that data. In this lab, that is you.

3.5 Controlled comparison and the label budget

Task 4 compares three things under the same conditions: the majority-class baseline, a convolutional network trained from scratch, and the transfer model with a frozen backbone.

- The design is the controlled experiment from Lab 2. The only factor that varies is **where the backbone weights came from**; split, seed, epochs, batch size, learning rate, and metric are all fixed.
- Calling `set_seed()` before building each model is what gives the two models the same starting point. Leave it out and the numbers depend on whatever used the random generator before the call. A hidden test checks exactly that.
- With K classes of roughly equal size, the majority-class baseline sits near $1/K$. Report parameter counts next to accuracy: on accuracy alone the two models look equally expensive.

Assumptions and limits:

- Transfer learning pays off **when labels are scarce**. As labels are added, a network trained from scratch has more to learn its own features from, and the advantage of borrowed features shrinks. Where that happens in this lab is what report Figure 2 measures.
- One split is one measurement. With this few labels, changing only the initial weights can move the from-scratch accuracy by ten points or more, so read a small gap with care. The backbone here was also pretrained on shapes from the same generator, which makes transfer work better than it would in a real setting.

4 Worked example: a 3 by 3 kernel on a 5 by 5 input

Work through the input and kernel in Table 2, with stride 1 and padding 1. The input is a bright vertical bar and the kernel is a vertical-edge filter.

Table 2. The input and kernel of the worked example. Cells holding 3 form the bright bar. The kernel has +1 down its left column and −1 down its right, so it responds to vertical edges.

0	0	0	0	0
0	3	3	0	0
0	3	3	0	0
0	3	3	0	0
0	0	0	0	0

input 5×5

1	0	−1
1	0	−1
1	0	−1

kernel 3×3

Step 1. Output size:

1. Put it in the formula: $\lfloor (5 + 2 \times 1 - 3)/1 \rfloor + 1 = \lfloor 4 \rfloor + 1 = \mathbf{5}$. Height and width give the same value, so the output is 5×5: a 3 by 3 convolution with padding 1 preserves the size.

Step 2. The two edges of the bar: output [2, 2] and output [2, 1]:

1. The window behind output [2, 2] is rows 1–3, columns 1–3, and its values are $(3, 3, 0)$ three times. Summing $3 \cdot 1 + 3 \cdot 0 + 0 \cdot (-1)$ over three rows gives **9**.
2. The window behind output [2, 1] is rows 1–3, columns 0–2, with values $(0, 3, 3)$ three times. Summing $0 \cdot 1 + 3 \cdot 0 + 3 \cdot (-1)$ over three rows gives $\mathbf{-9}$.
3. The same kernel returns a negative value at the left edge of the bar and a positive one at the right edge. The sign encodes direction.

Step 3. A cell that reaches into the padding: output [0, 0]:

1. The top row and the left column of this window are padding, so they are zero. The other four cells are the top-left 2×2 corner of the input, and only one of them holds 3. It lands in the bottom-right corner of the window, where the kernel value is -1, so the output is $3 \times (-1) = \mathbf{-3}$.
2. This window covers only one cell of the bar, so its response is -3 rather than -9. Where a pattern runs into the border, the zero padding pulls the response towards zero.

Step 4. The flattened dimension of this network:

1. Start at 32 and alternate convolution and pooling. Convolution with padding 1 preserves the size, so only the pooling counts: $32 \to 16 \to 8 \to 4$.
2. The last convolution has 32 channels, so the feature map is $32 \times 4 \times 4$, which flattens to $32 \times 4 \times 4 = \mathbf{512}$. The head is `Linear(512, 4)`.

5 Common mistakes

1. **Getting the flattened dimension wrong.** A wrong input dimension on the head raises a shape error on the first forward pass. Write the size down at every layer. Table 1 lists the size after every stage.
2. **Checking output sizes by building a layer.** Reading the shape off a dummy tensor exhausts memory on large inputs, and a hidden test uses a size that cannot be allocated.
3. **Forgetting to freeze.** Leave `requires_grad` alone and the backbone trains too. Nothing raises an error; only the numbers change. A hidden test compares the backbone weights before and after training.
4. **Trying to load the saved head as well.** The six-class head does not fit, so `load_state_dict` raises. Select only the keys that start with `features.` and load them into `model.features`.
5. **Using different conditions per model.** Give one model more epochs, or re-split between them, and you cannot tell which change produced the difference. Split once and use that split for both. Skipping `set_seed()` before each model is the same mistake.

6 How this maps to the lab tasks

Table 3. Tasks matched to concept sections. Which section each task draws on, and what grading looks at.

Lab task	Concept	What grading looks at
Task 1. Output size	§3.1, §4	floor division, height and width separately
Task 2. Build the CNN	§3.2, §3.3	layer types and order, flattened dimension 512
Task 3. Freeze and replace	§3.4	`features` only, correct `requires_grad`
Task 4. Compare two models	§3.5	one factor varies, numbers that reproduce
Report figures 1–3	§3.4, §3.5	the data, the gain from labels, the error structure

7 Self-check

1. The input is 28×28, the kernel 5×5, stride 2, padding 0. What is the output size, and how many border cells are discarded?
2. Use the formula to explain why padding 1 preserves the size for a 3 by 3 convolution. What padding would a 5 by 5 kernel need?
3. Remove the last pooling layer. What is the flattened dimension then, and how many parameters does the head have?
4. You froze the backbone, yet its weights changed after training. What do you check first?
5. With 5,000 labels instead of 240, would you pick the transfer model or the from-scratch model? On what evidence?

These questions are fair game for the oral check. You must be able to explain every line you submit.

8 Further reading

- PyTorch Tutorials, Transfer Learning for Computer Vision `https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html`
- Dumoulin & Visin, *A Guide to Convolution Arithmetic for Deep Learning*, 2016 `https://arxiv.org/abs/1603.07285`
- Yosinski et al., *How Transferable Are Features in Deep Neural Networks?*, 2014 `https://arxiv.org/abs/1411.1792`